

1
Week

4
Day

5
Topics

Error Handling &
Modules
Focus

Topic 1 — try / catch / finally & Custom Error Classes

Error handling production-ready code ka sabse important part hai. **try** block mein risky code likhte hain. **catch** block error pakadta hai. **finally** hamesha run hota hai — chahe error aaye ya na aaye. JavaScript mein built-in Error types hain: Error, TypeError, RangeError, ReferenceError, SyntaxError. Aap **custom error classes** bhi bana sakte ho — better debugging ke liye.

try {} Risky code yahan likhte hain — jo throw kar sakta ho.	catch(e) {} Error pakadta hai — e mein error object hota hai (message, name, stack).	finally {} Cleanup code — hamesha run hota hai, chahe error ho ya na ho.	throw Manually error throw karo — any value throw ho sakti hai, object best practice.
--	--	--	---

try / catch / finally — Basics & Real Pattern

```
// Basic try/catch/finally
function divideNumbers(a, b) {
  try {
    if (typeof a !== 'number' || typeof b !== 'number')
      throw new TypeError('Both arguments must be numbers');
    if (b === 0)
      throw new RangeError('Cannot divide by zero');
    return a / b;
  } catch (err) {
    console.error(`[${err.name}] ${err.message}`);
    return null; // graceful fallback
  } finally {
    console.log('divideNumbers() call finished'); // always runs!
  }
}

divideNumbers(10, 2); // 5 + 'call finished'
divideNumbers(10, 0); // [RangeError] Cannot divide by zero + null
divideNumbers('a', 2); // [TypeError] Both arguments... + null

// finally real use – cleanup resources
async function readFile(path) {
  let fileHandle = null;
  try {
    fileHandle = await openFile(path);
    return await fileHandle.read();
  } catch (err) {
```

```
console.error('Read failed:', err.message);
throw err; // re-throw after logging
} finally {
if (fileHandle) await fileHandle.close(); // always close!
}
}
```

Custom Error Classes — Hierarchy

[illegible]

```

return { success: false, field: err.field, message: err.message };
}
throw err; // unknown errors re-throw karo
}
}

processUser({ name: 'Rahul', age: 16 });
// { success: false, field: 'email', message: 'Email required' }

```

■ Critical Error Handling Mistakes

1. Empty catch: `catch(e){}` — error silently swallow karna. NEVER do this!
2. `catch(e){ console.log(e) }` — log karo lekin throw bhi karo agar handle nahi kar sakte.
3. `typeof` check miss karna — throw statement se pehle validate karo.
4. finally mein return — catch ka return override kar deta hai! Confusing bug.
5. Error ka message generic rakhna: 'Something went wrong' — useless in production.

Topic 2 — ES Modules: import / export (Named & Default)

ES Modules (ESM) JavaScript ka official module system hai — ES2015 mein introduce hua. Modules se code ko separate files mein organize kar sakte hain — har file ka apna scope hota hai, nothing leaks to global. **Named exports** — multiple exports per file. **Default export** — ek file se ek primary thing export.

Named Export

```

// math.js – Named Exports
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}

export class Vector {
  constructor(x, y) {
    this.x = x; this.y = y;
  }

  magnitude() {
    return Math.sqrt(this.x**2+this.y**2);
  }
}

// NOT exported – private to this file
function _helper() { return 42; }

```

Named Import

```

// main.js – Named Imports
import { PI, add, multiply } from './math.js';

// Rename on import
import { add as sum } from './math.js';

// Import everything as namespace
import * as Math2 from './math.js';
Math2.add(2, 3); // 5
Math2.PI; // 3.14159

// Import class
import { Vector } from './math.js';
const v = new Vector(3, 4);
v.magnitude(); // 5

// Curly braces = named import
// No curly braces = default import

```

Default Export

```
// logger.js – Default Export
class Logger {
  constructor(prefix = 'App') {
    this.prefix = prefix;
  }
  log(msg) {
    console.log(`[${this.prefix}] ${msg}`);
  }
  error(msg) {
    console.error(`[${this.prefix}] ERROR: ${msg}`);
  }
}

// Only ONE default export per file
export default Logger;

// Mix: default + named
export const LOG_LEVELS =
['info', 'warn', 'error'];
```

Default Import

```
// main.js – Default Import
// No curly braces for default
import Logger from './logger.js';

// Name it ANYTHING you want
import MyLogger from './logger.js';
import Log from './logger.js';

// Mix default + named in one line
import Logger, { LOG_LEVELS } from
'./logger.js';

const log = new Logger('API');
log.log('Server started');

// [API] Server started
LOG_LEVELS; // ['info', 'warn', 'error']
```

Barrel File Pattern — Re-exports

```
// Re-exporting – index.js as barrel file
// utils/index.js – single entry point for whole utils folder
export { add, multiply, PI } from './math.js';
export { default as Logger } from './logger.js';
export { validateEmail } from './validators.js';

// Consumer imports from ONE place
import { add, Logger, validateEmail } from './utils/index.js';
// Instead of importing from 3 different files!

// Rename on export
export { internalFn as publicFn } from './internal.js';
```

■ Named vs Default — Common Confusion

Named: `import { add } from './math'` — curly braces, exact name match karna padta hai.

Default: `import add from './math'` — no braces, aap koi bhi naam de sakte ho.

Ek file mein ek hi default export ho sakta hai — multiple named exports ho sakte hain.

`import React from 'react'` — default. `import { useState } from 'react'` — named.

Refactoring trap: named export rename karo to sab imports update karne padte hain.

Topic 3 — CommonJS require() vs ES Modules (ESM)

CommonJS (CJS) Node.js ka original module system hai — synchronous, runtime loading. **ES Modules** browser aur modern Node.js ka standard hai — static, async, tree-shakable. Dono alag hain — ek project mein mix karna tricky ho sakta hai.

```
// CommonJS (CJS) – Node.js default
// Export

const PI = 3.14;

function add(a,b) { return a+b; }

class Calc { }

module.exports = { PI, add, Calc };

// OR single export:

module.exports = add;

// Import (require)

const math = require('./math');

const { add, PI } = require('./math');

// Dynamic require OK

const mod = require(condition
? './modA' : './modB');

// __dirname, __filename available

console.log(__dirname);
```

```
// ES Modules (ESM) – Modern standard
// Export

export const PI = 3.14;

export function add(a,b) { return a+b; }

export class Calc { }

export default add;

// Import (static – top of file only)

import math from './math.js';

import { add, PI } from './math.js';

// Dynamic import() – returns Promise

const mod = await import(condition
? './modA.js' : './modB.js');

// __dirname NOT available in ESM

// use import.meta.url instead
```

CJS: synchronous (blocks).
ESM: async (non-blocking, better for browsers).

CJS: not tree-shakable.
ESM: bundlers remove unused exports automatically.

CJS: `require()` anywhere in code. ESM: `import` must be at TOP LEVEL (static).

- .cjs = CommonJS always.
- .mjs = ESM always. .js = depends on package.json type field.

New projects: ESM use karo — standard hai, Vite/webpack/Node 20+ sab support karte hain.

Legacy Node.js code: CJS theek hai — require() abhi bhi kaam karta hai.

package.json mein 'type': 'module' set karo to .js files ESM treat hongi.

Interop: ESM CJS require() nahi kar sakta directly (use createRequire). CJS dynamic import() se ESM load kar sakta hai.

Dynamic import() ek function hai jo Promise return karta hai — module runtime pe load hota hai, on-demand. Static import hamesha file load hone pe run hota hai. Dynamic import se **code splitting** aur **lazy loading** possible hoti hai — initial bundle size reduce hota hai, app faster load hota hai.

[illegible]

```

new Chart('#canvas').render(data);
}

// Conditional loading – load only what's needed
async function loadLocale(lang) {
  const locale = await import(`./locales/${lang}.js`);
  return locale.default; // default export access
}

loadLocale('hi'); // loads ./locales/hi.js only
loadLocale('en'); // loads ./locales/en.js only

// Lazy load on user action (React-like pattern)
button.addEventListener('click', async () => {
  const { openModal } = await import('./modal.js');
  openModal(); // modal code loads only when button clicked!
});

// Load multiple modules in parallel
async function loadDashboard() {
  const [{ Chart }, { Table }, { Map }] = await Promise.all([
    import('./chart.js'),
    import('./table.js'),
    import('./map.js'),
  ]);
}

// All three load in parallel – faster!

// With error handling
async function safeImport(path) {
  try {
    const module = await import(path);
    return module;
  } catch (err) {
    console.error(`Failed to load module: ${path}`, err);
    return null;
  }
}

```

■ React.lazy() — Dynamic Import in React

React.lazy(() => import('./HeavyComponent')) — component level lazy loading.

Wrap in Suspense: loading fallback dikha sakte ho jab module load ho raha ho.

Route-based code splitting: har page ka bundle alag — only load when user visits that route.

Vite/webpack automatically split chunks on dynamic import — no config needed!

Topic 5 — Module Bundling Concept (Why Webpack / Vite?)

Browser 100+ separate HTTP requests nahi kar sakta efficiently. **Module bundler** (Webpack, Vite, Rollup, Parcel) sab JS files ko ek ya kuch bade files mein combine karta hai — **bundle**. Additionally: TypeScript transpile, CSS process, images optimize, tree shaking, hot module replacement (HMR) bhi karta hai.

Bundling 100+ files → 1-3 optimized bundles. Network requests kam. Parse time kam.	Tree Shaking Unused exports automatically remove — bundle size chota. ESM se hi possible.	Code Splitting Bundle ko chunks mein split — lazy load, initial load faster.	HMR Hot Module Replacement — save karo, browser instantly update bina full reload.
--	---	--	--

Bundling Concepts — Vite Example

```
// Without bundler – browser 5 requests karta hai:
// GET /utils.js
// GET /validators.js
// GET /api.js
// GET /components.js
// GET /app.js

// With bundler – browser sirf 1-2 requests:
// GET /dist/bundle.js (all JS combined)
// GET /dist/vendor.js (3rd party libs separate chunk)

// Vite config (vite.config.js) – minimal setup
import { defineConfig } from 'vite';
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'], // separate vendor chunk
        }
      }
    }
  }
});

// Tree shaking in action:
// math.js exports: add, subtract, multiply, divide, factorial, fibonacci
import { add } from './math.js'; // only add is used
// Bundler removes subtract/multiply/divide/factorial/fibonacci from bundle!
// Bundle smaller by removing dead code

// Dynamic import = automatic code split point
const AdminPanel = React.lazy(() => import('./AdminPanel'));
// Bundler creates: chunk-AdminPanel.js – loaded only for admin routes
```

Webpack (older)

```
// Webpack – older, more config
// webpack.config.js
module.exports = {
  entry: './src/index.js',
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: 'bundle.js'
  },
  module: {
    rules: [
      { test: /\.jsx?$/, use: 'babel-loader' },
      { test: /\.css$/, use: ['style-loader', 'css-loader'] }
    ]
  }
};
// More verbose – full control
```

Vite (modern — we use this)

```
// Vite – modern, fast, minimal config
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  // That's it for React! Zero extra config.
});
// Dev: uses native ESM – no bundle!
// Prod: uses Rollup for optimized bundle
// HMR: instant updates in <50ms
// Why course uses Vite!
```

Day 4 Learning Outcomes

try/catch/finally confidently likhna aayega — cleanup pattern bhi.

Custom Error classes extend Error se banana aayega — statusCode, field, etc.

Named aur default exports/imports confidently use kar paoge.

CommonJS aur ESM ka farq samjha aayega — kab kya use karen.

Dynamic import() se lazy loading implement kar paoge.

Vite/Webpack kyun use hote hain — bundling concepts clear honge.